

Lab 3 — Transforming Space

Friday, October 9

Last updated: October 5, 2026.

Date is on the [schedule](#). Same day: [Quiz 5](#) (basis — not transforms) and a short [Lecture 20](#), then this lab.

Credit: Lab points are for effort in class, not for a Canvas file upload. Work the hub exercises in Octave or Python on a laptop. On lab day you get a **self-contained one-pager** with the exercises written out, space for answers and new syntax, and a blank for instructor initials after a short check-in. Preview the [one-pager](#); write on the hard copy handed out in class. Quizzes will ask about these ideas ([Quiz 6](#) covers Week 7).

Resources: Lachniet Ch. 2 · [Syntax sheets](#) · [Resources](#) · Lectures [18–20](#).

What we are doing

Treat a matrix as a geometric machine: predict where points go under scaling, reflection, shear, and rotation; apply A to many points at once; plot before/after.

Learning objectives

1. Predict the effect of scaling, reflection, shear, and rotation matrices.
2. Apply a matrix to many 2D points stored as columns (or rows) of a data matrix.
3. Plot original and transformed shapes and compare.
4. Adapt a short program that implements a transformation.
5. Interpret a matrix as a coordinate or physical-space transformation.

Math connections

Matrix dimensions and structure ([Lecture 18](#)); Ax as a linear combination of columns ([Lecture 19](#)); geometric maps and “columns = images of e_j ” ([Lecture 20](#)).

Key syntax (this lab)

Task	Octave	Python
Matrix	<code>A = [1 0; 0 -1]</code>	<code>A = np.array([[1., 0.], [0., -1.]])</code>
Matrix–vector	<code>A*x</code>	<code>A @ x</code>
Many points as columns	<code>Y = A*X</code>	<code>Y = A @ X</code>
2D scatter / line plot	<code>plot(X(1,:), X(2,:), 'o-')</code>	<code>plt.plot(X[0], X[1], 'o-')</code>

Indexing reminder: Boyd A_{ij} is 1-based. Octave matrices are 1-based; NumPy is 0-based.

Demo (optional — run once)

Triangle with vertices $(0,0)$, $(1,0)$, $(0,1)$ as columns of X , then rotate 90° CCW by $A = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$.

Octave:

```

X = [0 1 0; 0 0 1];           % columns are points
A = [0 -1; 1 0];
Y = A*X
plot(X(1,:), X(2,:), 'bo-', Y(1,:), Y(2,:), 'rs-');
axis equal; legend('original','transformed');

```

Python:

```

import numpy as np
import matplotlib.pyplot as plt
X = np.array([[0., 1., 0.], [0., 0., 1.]]) # columns are points
A = np.array([[0., -1.], [1., 0.]])
Y = A @ X
plt.plot(X[0], X[1], 'bo-', Y[0], Y[1], 'rs-')
plt.axis('equal'); plt.legend(['original', 'transformed']); plt.show()

```

Expected: $Y = \begin{bmatrix} 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}$ (columns $(0,0)$, $(0,1)$, $(-1,0)$).

Exercises

Pick Octave or Python for the term. Store points as **columns** of a $2 \times N$ matrix unless a prompt says otherwise.

E1. Predict, then compute. Let $A = \text{diag}(2, 1/2)$ and $x = (1, 1)$.

- (a) Predict Ax without software.
- (b) Check with code.
- (c) In one sentence: what did A do geometrically?

Octave: `A = diag([2, 0.5]); x = [1; 1]; A*x`

Python: `A = np.diag([2.0, 0.5]); x = np.array([1.0, 1.0]); A @ x`

Expected: $(2, 0.5)$; horizontal stretch by 2, vertical shrink by $1/2$.

E2. Build from e_1, e_2 . Reflection through the x_2 -axis sends $(1, 0) \mapsto (-1, 0)$ and $(0, 1) \mapsto (0, 1)$.

- (a) Write A on paper.
- (b) Apply A to the three columns of $X = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$.
- (c) Sketch or plot original vs image.

Expected: $A = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$; $AX = \begin{bmatrix} -1 & 0 & -1 \\ 0 & 1 & 1 \end{bmatrix}$.

E3. Shear. Let $A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ and unit-square corners as columns

$$X = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}.$$

Compute $Y = AX$. Which corner moved, and how? (Hand-check Ae_2 .)

Expected: $Y = \begin{bmatrix} 0 & 1 & 2 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$; $(0, 1) \mapsto (1, 1)$ (and $(1, 1) \mapsto (2, 1)$) — sheared right.

E4. Rotation. Use $A = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$ (90° CCW) on the same unit-square X from E3. Compute $Y = AX$ and plot both. Does the square stay a square?

Expected: $Y = \begin{bmatrix} 0 & 0 & -1 & -1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$; yes — rotated square (still a square).

E5. Adapt the program. Change E4's matrix to rotation by 180° (or use $A = -I$). Apply to X from E3. Where does $(1, 0)$ go?

Expected: $(1, 0) \mapsto (-1, 0)$; full $Y = \begin{bmatrix} 0 & -1 & -1 & 0 \\ 0 & 0 & -1 & -1 \end{bmatrix}$.

E6 (stretch / 3D idea). Let

$$Q = \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

(level flight heading south; Boyd-style body-to-earth axes). Compute Qe_1 , Qe_2 , Qe_3 and interpret each column as “where that body axis points in earth coordinates.”

Expected: columns $(0, -1, 0)$, $(1, 0, 0)$, $(0, 0, 1)$ — forward→south, left→east, up→up.

One-pager

Preview the [Lab 3 ticket](#); write on the hard copy in class.